

Exercise 01 — Exploration and analysis

Goal: produce a table of genes that differ between the control and treated conditions, and be able to say why you trust each step that produced it.

Expected outcome:

- Results in `results/de_results.tsv`, contains a column of adjusted p-values
- You can answer the three questions in **Checkpoint C** without looking anything up.

CONTENTS

Before you start

The data

Step 1. Ask before you build

Step 2. Quality assessment

Step 3 and 4. Filter and normalize

Step 5 and 6. Test, and correct for multiple testing

Extension (using below as prompts to Claude)

Before you start

Open a terminal to access the "Data_Analysis" directory and start Claude Code:

```
cd Data_Analysis
claude
```

Once Claude Code is running you do not need a second terminal. Anything you type beginning with `!` is run as a shell command instead of being sent to the model, and its output appears in the conversation where Claude can see it as well:

```
! python3 selfcheck.py
```

Every command in this handout is written that way. Drop the `!` if you would rather run it in a separate terminal window.

Two more words that will be used throughout: **Context** is everything Claude can currently see: the files it has read and the conversation so far. **Plan mode** is a state in which Claude describes what it intends to do without changing any files. Press `Shift+Tab` to enter it.

The data

FILE	WHAT IT HOLDS
<code>data/counts.tsv</code>	4200 genes by 16 samples. Raw counts, plus gene id, symbol and length.
<code>data/sample_metadata.tsv</code>	One row per sample: condition, replicate number, batch.

Eight control samples and eight treated samples. The counts are synthetic. They were generated from a model with a known answer, which you will use in Exercise 02.

Step 1. Ask before you build

Start with this prompt:

PROMPT

I have a bulk RNA-seq experiment in this folder. `data/counts.tsv` is a raw count matrix and `data/sample_metadata.tsv` describes the samples. I want to find the genes that differ between the control and treated conditions. Read both files first. Then describe what you found and the analysis steps you propose, in order, with a reason for each one. Do not write any code yet.

Read what comes back before you reply. You are checking three things:

1. Does the number of genes and samples it reports match the table above?
2. Does its proposed plan mention the processing batch?
3. Does it propose a method for adjusting p-values for multiple testing?

If any of the three is missing, say so and ask Claude to revise the plan.

Step 2. Quality assessment

PROMPT

Now write and run a script that checks the quality of this dataset. I want the library size and the number of detected genes per sample, a principal component analysis of the samples, and a sample to sample correlation figure. Color the points by condition and use a different symbol for each batch. Save the figures into `results/figures/` and show me the numbers.

Library size is the total number of counts in a sample. **Principal component analysis** places each sample on a small number of axes chosen so that the first axis captures as much of the variation between samples as possible.

When the figures exist, open `results/figures/02_pca.png` and answer:

- Which variable separates the samples along the first component?
- Which variable separates them along the second?

If you cannot tell from the figure, ask Claude to label the points.

Checkpoint A

Can you state the number of genes, the number of samples, and which variable dominates the first principal component?

Behind? Run `! python3 checkpoints/01_quality_assessment.py` and continue.

Step 3 and 4. Filter and normalize

PROMPT

Filter out genes with too few counts to be tested, then normalize the counts so that samples can be compared. Tell me which filtering threshold you chose and how many genes it removes. Tell me which normalization method you chose, why that one, and what the alternatives are. Save the normalized matrix to `results/normalized_counts.tsv`.

Twelve genes in this dataset carry about a fifth of all the counts, in the way that ribosomal and mitochondrial transcripts do in a real experiment. This makes the choice of normalization method matter. Ask directly:

PROMPT

Would my result change if I had normalized by total count per sample instead? Show me the difference rather than describing it.

Normalization puts samples on a comparable scale so that a sample sequenced more deeply does not appear to have higher expression everywhere.

Checkpoint B

`results/normalized_counts.tsv` exists, and can you name the normalization method that produced it?

Behind? Run `! python3 checkpoints/02_filtered_and_normalized.py` and continue.

Step 5 and 6. Test, and correct for multiple testing

PROMPT

Test each gene for a difference between the treated and control conditions. Correct the p-values for multiple testing. Write the results to `results/de_results.tsv` with the columns `gene_id`, `gene_symbol`, `mean_expression`, `log2_fold_change`, `pvalue` and `padj`. Then tell me how many genes pass an adjusted p-value below 0.05 together with an absolute log2 fold change of at least 1. Save the whole analysis, from loading the counts to writing that table, as one script called `analysis_versionControl.py` that I can run again from the command line.

About 3900 genes are tested. At an unadjusted threshold of 0.05, roughly 195 genes would be called significant even if no gene differed at all. **Multiple testing correction** accounts for this. **Adjusted p-value**, written `padj`, is the corrected value. **log2 fold change** is the difference on a log2 scale: a value of 1 means the gene is twice as high in the treated samples.

Then ask the model to check itself:

PROMPT

List the assumptions in this analysis. State what would make the result incorrect. State what you did not check.

Keep that answer on screen.

Keep the script, not the conversation with Claude

`analysis_versionControl.py` is now the only version of your analysis that will still exist next month. The conversation will not, and the same prompt run twice gives different code. First check that the script stands on its own:

```
! python3 analysis_versionControl.py
```

It must rebuild `results/de_results.tsv` without you typing anything else. Then put it under version control:

```
! git init                                # only if you downloaded the ZIP
! git add analysis_versionControl.py
! git commit -m "analysis as run"
```

No git on your machine? Skip the three commands and carry on. Everything else in the session works without them.

Checkpoint C

`results/de_results.tsv` exists, `analysis_versionControl.py` reruns on its own, and you can answer:

1. How many genes were tested, and how many were removed by filtering?
2. Which statistical test was used, and which correction method?
3. Did the analysis use the processing batch anywhere?

Behind? Run `! python3 checkpoints/03_differential_expression.py` and continue. It writes the table but not the script, so you will not have done the version control step — come back to it if there is time.

Extension (using below as prompts to Claude)

For those who finish early. Each one is worth doing on its own.

1. **Add the batch to the model.** The analysis so far compares two conditions and ignores the processing batch. Fit a linear model with both terms and compare the two gene lists. How many genes appear in one and not the other?
2. **Compare two normalization methods.** Run the whole analysis with counts per million and again with median of ratios. Quantify the difference in the result rather than describing it.
3. **Test the stability of the gene list.** Remove one sample at a time, rerun, and record how often each gene remains in the top 100. Genes that drop out when a single sample is removed are worth less confidence.